Tensoren und Tensorprodukte

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- **Lineare Algebra**
  - Vektorräume und Unterräume
  - Basis und Dimension
  - Lineare Abbildungen und Multilinearität
  - Äußeres Produkt: $\boldsymbol{v} \otimes \boldsymbol{w} = \boldsymbol{v}\boldsymbol{w}^T$
  - $\operatorname{span}\left(\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}\right) = \{\boldsymbol{w} : \boldsymbol{w} = \sum_i c_i \boldsymbol{v}_i; c_i \in \mathbb{R}\}$
- **Analysis**
  - Multivariate Funktionen $f(\boldsymbol{x}) = f(x_1, x_2, \ldots, x_n)$
- **Gleichungssysteme** (Kapitel 5): LGS $\boldsymbol{M}\boldsymbol{z} = \boldsymbol{c}$ lösen (LU, Cholesky)

Multilineare Abbildungen

Multilineare Abbildungen

- Bisher wurde ein Vektor $\boldsymbol{v} \in V$ linear auf einen Vektor $\boldsymbol{w} \in W$ abgebildet.
- Jetzt werden Paare oder Tupel von Vektoren $(\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n) \in V_1 \times \cdots \times V_n$ “linear” auf ein $\boldsymbol{w} \in W$ abgebildet,
  - $\rightarrow$ Wir sprechen von einer *multilinearen Abbildung*.
- Linearität bezieht sich hier auf die *einzelnen* Argumente der Abbildung.

Multilineare Abbildungen

Def.: Multilineare Abbildung

Seien $V_1, \dots, V_n, W$ Vektorräume. Eine **multilineare Abbildung** ist eine Abbildung $f\colon V_1 \times \cdots \times V_n \to W$, sodass:

$$\begin{aligned} &f(\boldsymbol{v}_1, \dots, c\boldsymbol{v}_i + c'\boldsymbol{v}_i', \dots, \boldsymbol{v}_n) \\ &= cf(\boldsymbol{v}_1, \dots, \boldsymbol{v}_i, \dots, \boldsymbol{v}_n) + c' f(\boldsymbol{v}_1, \dots, \boldsymbol{v}_i', \dots, \boldsymbol{v}_n) \end{aligned}$$

für alle $i = 1, \dots, n$, $c, c' \in \mathbb{R}$, $\boldsymbol{v}_i, \boldsymbol{v}_i' \in V_i$ und $\boldsymbol{v}_\ell \in V_\ell$ ($\ell \neq i$).

Anmerkungen:

- Im Spezialfall $n = 2$ nennen wir die Abbildung *bilinear*.
- **Achtung:** *Multilineare Abbildungen* sind in der Regel nicht linear!

Intuition: Linear vs. Multilinear

Fläche eines Rechtecks als Funktion seiner Seitenlängen:

$$f\colon \mathbb{R} \times \mathbb{R} \to \mathbb{R}, \quad (x, y) \mapsto xy$$

Warum ist f nicht linear?

- Für $c = 2$: $f(2x, 2y) = 2x \cdot 2y = 4xy = 4f(x, y) \neq 2f(x, y)$ (für $xy \neq 0$)
- Linear wäre: $f(cx, cy) = c \cdot f(x, y)$

Aber f ist bilinear:

$$\begin{aligned} y \text{ konstant: } f(x_1 + x_2, y) &= (x_1 + x_2) \cdot y = x_1 y + x_2 y = f(x_1, y) + f(x_2, y) \\ f(cx, y) &= (cx) \cdot y = c(xy) = cf(x, y) \\ x \text{ konstant: } f(x, y_1 + y_2) &= x \cdot (y_1 + y_2) = xy_1 + xy_2 = f(x, y_1) + f(x, y_2) \\ f(x, cy) &= x \cdot (cy) = c(xy) = cf(x, y) \end{aligned}$$

$\Longrightarrow$ Die Funktion ist **in jedem Argument einzeln linear**!

Multilineare Abbildungen

Beispiel:

- Multiplikation von zwei reellen Zahlen, $f\colon \mathbb{R} \times \mathbb{R} \to \mathbb{R}$, $(x_1, x_2) \mapsto x_1 x_2$, ist bilinear (s.a. Bsp. zur Fläche)

Beispiel:

- Jedes Skalarprodukt $f\colon V \times V \to \mathbb{R}$, $(\boldsymbol{v}_1, \boldsymbol{v}_2) \mapsto \langle \boldsymbol{v}_1, \boldsymbol{v}_2 \rangle$ ist bilinear.
- Das folgt aus Distributivgesetz, Homogenität und Symmetrie:
  $\langle c\boldsymbol{v} + c'\boldsymbol{v}', \boldsymbol{w} \rangle = \langle c\boldsymbol{v}, \boldsymbol{w} \rangle + \langle c'\boldsymbol{v}', \boldsymbol{w} \rangle = c\langle \boldsymbol{v}, \boldsymbol{w} \rangle + c'\langle \boldsymbol{v}', \boldsymbol{w} \rangle$

Beispiel:

- Die Matrizenmultiplikation $\mathbb{R}^{m \times k} \times \mathbb{R}^{k \times n} \to \mathbb{R}^{m \times n}$ mit $(\boldsymbol{A}, \boldsymbol{B}) \mapsto \boldsymbol{AB}$ ist bilinear.

Quiz

TRUE/FALSE?

> Für jedes multilineare f gilt $f(\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n) = \mathbf{0}$ sobald $\boldsymbol{v}_i = \mathbf{0}$ für ein i.

Darstellung von multilinearen Abbildungen

Theorem

Eine Abbildung $T\colon \mathbb{R}^{n_1} \times \cdots \times \mathbb{R}^{n_k} \to \mathbb{R}^m$ ist genau dann multilinear, wenn es reelle Koeffizienten

$$A = (a_{i_1,\ldots,i_k,j}\colon 1 \leq i_1 \leq n_1, \ldots, 1 \leq i_k \leq n_k, 1 \leq j \leq m)$$

gibt, sodass

$$T(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(k)}) = \sum_{i_1=1}^{n_1} \cdots \sum_{i_k=1}^{n_k} \sum_{j=1}^{m} a_{i_1,\ldots,i_k,j} x_{i_1}^{(1)} \cdots x_{i_k}^{(k)} \boldsymbol{e}_j,$$

wobei $\boldsymbol{e}_j$ der j-te Einheitsvektor des $\mathbb{R}^m$ ist. Die Koeffizienten $a_{i_1,\ldots,i_k,j}$ sind eindeutig.

Tensoren

Tensoren

- Setzen wir im vorherigen Theorem $k = 1$, erhalten wir Koeffizienten $A = (a_{i,j} \colon 1 \leq i \leq n, 1 \leq j \leq m)$, sodass

$$T(\boldsymbol{x}) = \sum_{i=1}^{n} \sum_{j=1}^{m} a_{i,j} x_i \boldsymbol{e}_j = \sum_{i=1}^{n} x_i \boldsymbol{a}_i = \boldsymbol{A}\boldsymbol{x},$$

  mit $\boldsymbol{a}_i = (a_{i,1}, \ldots, a_{i,m})^\top \in \mathbb{R}^m$ und $\boldsymbol{A} = (\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n) \in \mathbb{R}^{m \times n}$, d.h. $(\boldsymbol{A})_{j,i} = a_{i,j}$.
- Die Ansammlung $\boldsymbol{A}$ aus nm Koeffizienten nennen wir *Matrix*.
- Sie hat zwei "Dimensionen" weil sie durch zwei Subskripte i und j indiziert wird.
- Allgemeiner beschreibt jede solche Ansammlung von Koeffizienten mit $k + 1$ Indizes (bei $m = 1$ also mit k) eine entsprechende multilineare Funktion

Bsp: Bilineare Abbildung ($k = 2, n_1 = n_2 = 2, m = 1$)

$T\colon \mathbb{R}^2 \times \mathbb{R}^2 \to \mathbb{R}$ mit $T(\boldsymbol{x}, \boldsymbol{y}) = 2x_1y_1 + 3x_1y_2 - x_2y_1 + 4x_2y_2$

Koeffizienten $(a_{i_1,i_2} \colon 1 \leq i_1, i_2 \leq 2)$**:**

$$a_{1,1} = 2, \quad a_{1,2} = 3, \quad a_{2,1} = -1, \quad a_{2,2} = 4 \implies \boldsymbol{A} = \begin{pmatrix} 2 & 3 \\ -1 & 4 \end{pmatrix}$$

Verifikation:

$$\begin{aligned} T(\boldsymbol{x}, \boldsymbol{y}) &= \sum_{i_1=1}^{2} \sum_{i_2=1}^{2} a_{i_1,i_2} x_{i_1} y_{i_2} = a_{1,1}x_1y_1 + a_{1,2}x_1y_2 + a_{2,1}x_2y_1 + a_{2,2}x_2y_2 \\ &= 2x_1y_1 + 3x_1y_2 - x_2y_1 + 4x_2y_2 \quad \checkmark \end{aligned}$$

Numerisches Beispiel:

Für $\boldsymbol{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \boldsymbol{y} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$: $T(\boldsymbol{x}, \boldsymbol{y}) = 2 \cdot 1 \cdot 3 + 3 \cdot 1 \cdot 4 - 1 \cdot 2 \cdot 3 + 4 \cdot 2 \cdot 4 = 6 + 12 - 6 + 32 = 44$

Alternativ in Tensorschreibweise:

$$T(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{y} = \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 2 & 3 \\ -1 & 4 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 0 & 11 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = 44$$

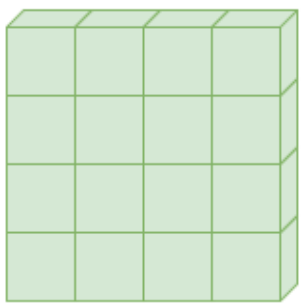
Definition: Tensor

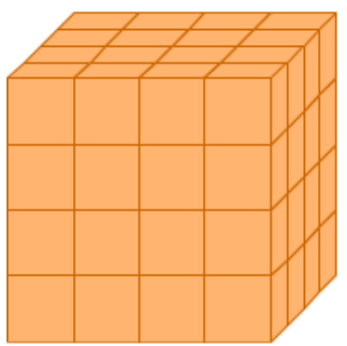
Def.: Tensor

Ein **Tensor** ist eine durch k Indizes indizierte Ansammlung von reellen Zahlen

$$A = (a_{i_1,\ldots,i_k} : 1 \leq i_1 \leq n_1, \ldots, 1 \leq i_k \leq n_k) \in \mathbb{R}^{n_1 \times \cdots \times n_k}.$$

Die Zahl k wird auch **Stufe** des Tensors genannt.





Eine (4×4)-Matrix besteht aus $4^2 = 16$ Zahlen (links); ein $(4 \times 4 \times 4)$-Tensor besteht aus $4^3 = 64$ Zahlen (rechts). (Quelle: Deisenroth/Faisal/Ong, Fig 5.13)

- Die Menge $\mathbb{R}^{n_1 \times \cdots \times n_k}$ aller $\left(n_1 \times \cdots \times n_k\right)$-Tensoren mit koeffizientenweiser Addition und Multiplikation mit Skalaren ist ein Vektorraum.

Anwendung: Tensoren in Deep Learning

- Eine Matrix stellt z.B. ein (Graustufen-)Bild dar: Dimension 1 indiziert die Pixel vertikal (Höhe), Dimension 2 horizontal (Breite).
- **RGB-Bild**: dritte Dimension indiziert die Farbkanäle $\Longrightarrow$ Stufe-3-Tensor $\boldsymbol{I} \in \mathbb{R}^{224\times224\times3}$ ($\approx 150 \cdot 10^3$ Werte pro Bild)
- **Batch/Video**: vierte Dimension indiziert mehrere Bilder — z.B. verarbeitet ein CNN 32 Bilder gleichzeitig $\Longrightarrow$ Stufe-4-Tensor $\boldsymbol{B} \in \mathbb{R}^{32\times224\times224\times3}$ ($\approx 4.8 \cdot 10^6$ Werte)
- **Feature Maps**: Convolutional Layers bilden Tensoren auf Tensoren ab: c: $\mathbb{R}^{32\times224\times224\times3} \to \mathbb{R}^{32\times56\times56\times64}$
- Rechnungen auf Tensoren können durch spezielle Hardware besonders schnell durchgeführt werden (Google TPUs, NVIDIA Tensor Cores, ...) — essentiell für KI.

(Details & Feature-Map-Visualisierung → Anhang)



$\tilde{=}$ 3 Kanal-Matrizen

$\to$



Produkt von Tensoren

Produkt von Tensoren

Def.: Äußeres Produkt

Das **äußere Produkt** von zwei Vektoren $\boldsymbol{v} \in \mathbb{R}^m$ und $\boldsymbol{w} \in \mathbb{R}^n$ ist definiert als

$$\boldsymbol{v} \otimes \boldsymbol{w} := \boldsymbol{v}\boldsymbol{w}^\top = \begin{pmatrix} v_1 w_1 & \cdots & v_1 w_n \\ \vdots & & \vdots \\ v_m w_1 & \cdots & v_m w_n \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

- *Äußeres Produkt*: $\otimes \colon \mathbb{R}^m \times \mathbb{R}^n \to \mathbb{R}^{m \times n}$
- vgl Skalarprodukt (“*inneres Produkt*”): $\langle \cdot, \cdot \rangle \colon \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$ $\langle \boldsymbol{v}, \boldsymbol{w} \rangle = \boldsymbol{v}^\top \boldsymbol{w}$

Äußeres Produkt: Beispiel & Eigenschaften

Beispiel:

$$\boldsymbol{v} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad \text{und} \quad \boldsymbol{w} = \begin{pmatrix} -2 \\ 3 \\ -11 \end{pmatrix}$$

$$\boldsymbol{v} \otimes \boldsymbol{w} = \boldsymbol{v}\boldsymbol{w}^\top = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \begin{pmatrix} -2 & 3 & -11 \end{pmatrix} = \begin{pmatrix} -2 & 3 & -11 \\ -4 & 6 & -22 \end{pmatrix}$$

- Alle Spalten sind Vielfache von $\boldsymbol{v}$, alle Zeilen sind Vielfache von $\boldsymbol{w}^\top$
  - Rang ist 1, sofern $\boldsymbol{v} \neq \boldsymbol{0}$ und $\boldsymbol{w} \neq \boldsymbol{0}$
- Was für eine lineare Abbildung ist $\boldsymbol{x} \to (\boldsymbol{v} \otimes \boldsymbol{w})\boldsymbol{x}$? (im Folgenden: $\boldsymbol{v} \neq \boldsymbol{0}$, $\boldsymbol{w} \neq \boldsymbol{0}$)
  - $(\boldsymbol{v} \otimes \boldsymbol{w})\boldsymbol{x} = \boldsymbol{v}\boldsymbol{w}^\top\boldsymbol{x} = \boldsymbol{v}\langle \boldsymbol{w}, \boldsymbol{x} \rangle$: Vektor $\boldsymbol{v}$ skaliert mit Skalarprodukt von $\boldsymbol{w}$ und $\boldsymbol{x}$
  - $\mathrm{col}(\boldsymbol{v} \otimes \boldsymbol{w}) = \mathrm{span}(\boldsymbol{v})$: Bild ist eindimensional
  - $\ker(\boldsymbol{v} \otimes \boldsymbol{w}) = \{\boldsymbol{x} : \boldsymbol{w}^\top \boldsymbol{x} = 0\}$: Hyperebene orthogonal zu $\boldsymbol{w}$

Anwendungsbeispiele:

- *SVD*: $\boldsymbol{A} = \boldsymbol{U}_r \boldsymbol{\Sigma}_r \boldsymbol{V}_r^\top = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \otimes \boldsymbol{v}_i$ (Darstellung einer Matrix als gewichtete Summe von normierten Rang-1-Matrizen)
- Statistik: $\mathrm{Cov}(\boldsymbol{x}) = \mathbb{E}\left[(\boldsymbol{x} - \mathbb{E}[\boldsymbol{x}])(\boldsymbol{x} - \mathbb{E}[\boldsymbol{x}])^\top\right] = \mathbb{E}\left[\boldsymbol{x} \otimes \boldsymbol{x}\right] - \mathbb{E}\left[\boldsymbol{x}\right] \otimes \mathbb{E}[\boldsymbol{x}]$
- KI: $\mathrm{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{1}{\sqrt{d_k}} \boldsymbol{Q}\boldsymbol{K}^\top\right)\boldsymbol{V}$

Tensorprodukt

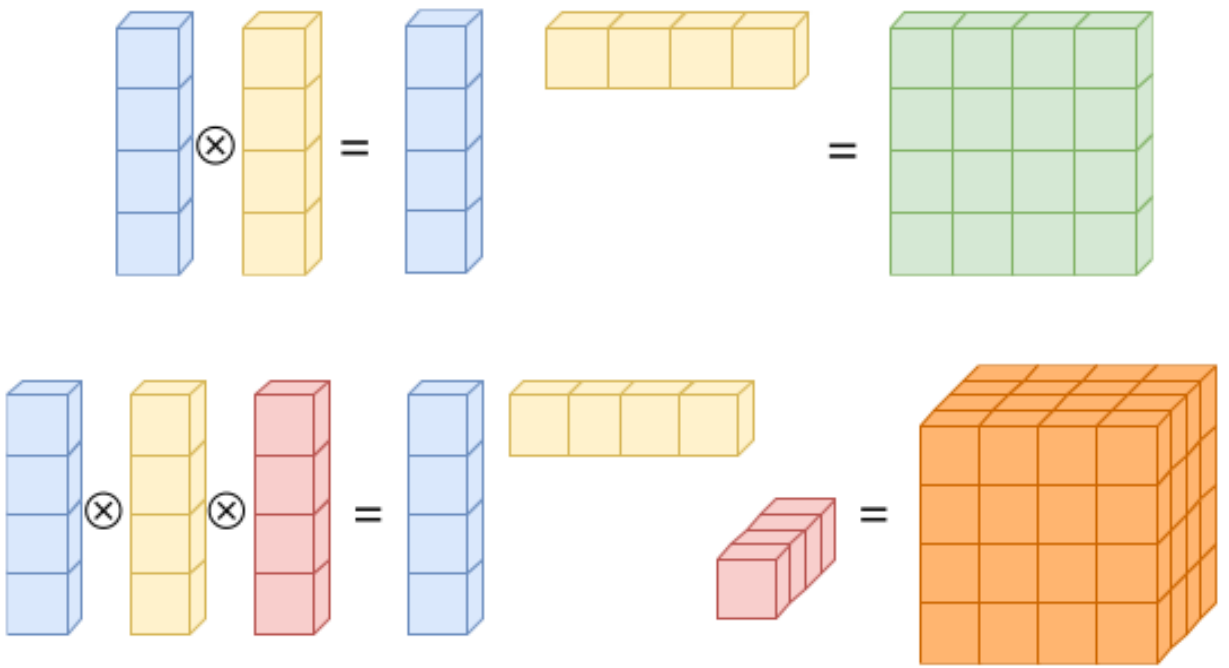
Def.: Tensorprodukt

Das **Tensorprodukt** $\otimes: \mathbb{R}^{m_1 \times \cdots \times m_p} \times \mathbb{R}^{n_1 \times \cdots \times n_q} \to \mathbb{R}^{m_1 \times \cdots \times m_p \times n_1 \times \cdots \times n_q}$ von zwei Tensoren $\boldsymbol{A} \in \mathbb{R}^{m_1 \times \cdots \times m_p}$ und $\boldsymbol{B} \in \mathbb{R}^{n_1 \times \cdots \times n_q}$ ist definiert als der Tensor $\boldsymbol{C} \in \mathbb{R}^{m_1 \times \cdots \times m_p \times n_1 \times \cdots \times n_q}$ mit

$$c_{i_1,\ldots,i_p,j_1,\ldots,j_q} = a_{i_1,\ldots,i_p} b_{j_1,\ldots,j_q}.$$

- *Äußeres Produkt* ist *Tensorprodukt* zweier Stufe-1-Tensoren ($p = q = 1$, also Vektoren)
- Das *Tensorprodukt* ist bilinear

Produkte von Tensoren



Oben: Das Produkt von zwei Stufe-1-Tensoren ergibt einen Stufe-2-Tensor. Unten: Das Produkt von drei Stufe-1-Tensoren ergibt einen Stufe-3-Tensor. Quelle: Deisenroth/Faisal/Ong, Fig 5.13

Bsp: Tensorprodukt

$$\boldsymbol{u} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \in \mathbb{R}^2, \quad \boldsymbol{v} = \begin{pmatrix} 3 \\ 5 \end{pmatrix} \in \mathbb{R}^2, \quad \boldsymbol{w} = \begin{pmatrix} 7 \\ 11 \end{pmatrix} \in \mathbb{R}^2$$

Tensorprodukt $\boldsymbol{T} = \boldsymbol{u} \otimes \boldsymbol{v} \otimes \boldsymbol{w} \in \mathbb{R}^{2\times 2\times 2}$ hat Einträge $\boldsymbol{T}_{i,j,k} = u_i v_j w_k$:

$$\begin{aligned}
\text{Slice 1 } (k=1, \text{ also } w_k = w_1 = 7)\text{:} \quad \boldsymbol{T}_{1,1,1} &= 1\cdot 3\cdot 7 = 21, & \boldsymbol{T}_{1,2,1} &= 1\cdot 5\cdot 7 = 35 \\
\boldsymbol{T}_{2,1,1} &= 2\cdot 3\cdot 7 = 42, & \boldsymbol{T}_{2,2,1} &= 2\cdot 5\cdot 7 = 70 \\
\text{Slice 2 } (k=2, \text{ also } w_k = w_2 = 11)\text{:} \quad \boldsymbol{T}_{1,1,2} &= 1\cdot 3\cdot 11 = 33, & \boldsymbol{T}_{1,2,2} &= 1\cdot 5\cdot 11 = 55 \\
\boldsymbol{T}_{2,1,2} &= 2\cdot 3\cdot 11 = 66, & \boldsymbol{T}_{2,2,2} &= 2\cdot 5\cdot 11 = 110
\end{aligned}$$

$$\implies \boldsymbol{T}_{\cdot,\cdot,1} = \begin{pmatrix} 21 & 35 \\ 42 & 70 \end{pmatrix}; \qquad \boldsymbol{T}_{\cdot,\cdot,2} = \begin{pmatrix} 33 & 55 \\ 66 & 110 \end{pmatrix}$$

Jede *slice* (Stufe-2-Tensor) dieses Stufe-3-Tensors ist eine Rang-1-Matrix:

- $\boldsymbol{T}_{\cdot,\cdot,1} = w_1 \cdot (\boldsymbol{u} \otimes \boldsymbol{v})$ und $\boldsymbol{T}_{\cdot,\cdot,2} = w_2 \cdot (\boldsymbol{u} \otimes \boldsymbol{v})$

Allgemein:
Ein Tensor der Form $\boldsymbol{u} \otimes \boldsymbol{v} \otimes \boldsymbol{w}$ hat *Rang 1* (für $\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w} \neq \boldsymbol{0}$).

Kroneckerprodukt

Def.: Kroneckerprodukt

Das **Kroneckerprodukt** von zwei Matrizen $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ und $\boldsymbol{B} \in \mathbb{R}^{p \times q}$ ist definiert als

$$\begin{aligned}
\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B} &:= \begin{pmatrix} a_{11}\boldsymbol{B} & \cdots & a_{1n}\boldsymbol{B} \\ \vdots & \vdots & \vdots \\ a_{m1}\boldsymbol{B} & \cdots & a_{mn}\boldsymbol{B} \end{pmatrix} \\
&= \begin{pmatrix} a_{11}b_{11} & \cdots & a_{11}b_{1q} & \cdots & a_{1n}b_{11} & \cdots & a_{1n}b_{1q} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{11}b_{p1} & \cdots & a_{11}b_{pq} & \cdots & a_{1n}b_{p1} & \cdots & a_{1n}b_{pq} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{m1}b_{11} & \cdots & a_{m1}b_{1q} & \cdots & a_{mn}b_{11} & \cdots & a_{mn}b_{1q} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{m1}b_{p1} & \cdots & a_{m1}b_{pq} & \cdots & a_{mn}b_{p1} & \cdots & a_{mn}b_{pq} \end{pmatrix} \in \mathbb{R}^{mp \times nq}.
\end{aligned}$$

Obacht – Notation:

$\otimes$ ist oft doppelt belegt (Tensorprodukt und Kroneckerprodukt) – wir schreiben ab hier $\otimes_{\mathrm{K}}$ fürs **Kroneckerprodukt**.

Kroneckerprodukt vs. Tensorprodukt

Kroneckerprodukt ist Variante des Tensorprodukts für Matrizen – **dasselbe Zahlenmaterial, andere Anordnung**:

Für $\boldsymbol{A} \in \mathbb{R}^{m \times n}, \boldsymbol{B} \in \mathbb{R}^{p \times q}$

- `Tensorprodukt` ergibt hier **Stufe-4-Tensor**

$$\boldsymbol{C} = \boldsymbol{A} \otimes \boldsymbol{B} \in \mathbb{R}^{m \times n \times p \times q} \quad \text{mit } c_{i_1,i_2,j_1,j_2} = a_{i_1,i_2} b_{j_1,j_2}$$

- `Kroneckerprodukt` ergibt **Matrix**

$$\boldsymbol{D} = \boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B} \in \mathbb{R}^{mp \times nq} \quad \text{mit } d_{(i_1-1)p+j_1,(i_2-1)q+j_2} = a_{i_1,i_2} b_{j_1,j_2}$$

- $\boldsymbol{C}$ und $\boldsymbol{D}$ unterscheiden sich (nur) darin wie ihre Einträge angeordnet sind – $\boldsymbol{D}$ ist *"abgeflachte"* Variante von $\boldsymbol{C}$

Notation:

- In der Literatur steht meist $\otimes$ für *beides* (!); welche Bedeutung gemeint ist, verrät dort das **Format** des Ergebnisses.
- In FMM ist $\otimes_{\mathrm{K}}$ immer das `Kroneckerprodukt`
- $\otimes$ bleibt `äußerem Produkt` bzw. `Tensorprodukt` vorbehalten.

> ⓘ **Hinweis**

Bsp: Kroneckerprodukt

Beispiel:

- Für

$$\boldsymbol{A} = \begin{pmatrix} 1 & 0 \\ 2 & 5 \end{pmatrix}, \qquad \boldsymbol{B} = \begin{pmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ -1 & 0 & -1 \end{pmatrix}$$

- Gilt

$$\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B} = \begin{pmatrix} 1 \cdot \boldsymbol{B} & 0 \cdot \boldsymbol{B} \\ 2 \cdot \boldsymbol{B} & 5 \cdot \boldsymbol{B} \end{pmatrix} = \begin{pmatrix} 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ -1 & 0 & -1 & 0 & 0 & 0 \\ 6 & 0 & 0 & 15 & 0 & 0 \\ 0 & 4 & 0 & 0 & 10 & 0 \\ -2 & 0 & -2 & -5 & 0 & -5 \end{pmatrix}$$

Vorsicht: $\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B} \neq \boldsymbol{B} \otimes_{\mathrm{K}} \boldsymbol{A}$! *Aber:* $(\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B})^\top = \boldsymbol{A}^\top \otimes_{\mathrm{K}} \boldsymbol{B}^\top$ – Transponieren dreht die Reihenfolge **nicht** um.

Quiz

Frage 1: Sei $\boldsymbol{A} \in \mathbb{R}^{3\times 2}$ und $\boldsymbol{B} \in \mathbb{R}^{2\times 4}$. In welchem Raum liegt das **Kroneckerprodukt** $\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B}$?

1. $\mathbb{R}^{6\times 8}$
2. $\mathbb{R}^{5\times 6}$
3. $\mathbb{R}^{3\times 2\times 2\times 4}$
4. $\mathbb{R}^{8\times 6}$

Frage 2: Sei $\boldsymbol{S} \in \mathbb{R}^{p\times p}$ eine Matrix mit Einträgen s_{ij}. Wie sieht das Kroneckerprodukt $\boldsymbol{S} \otimes_{\mathrm{K}} \boldsymbol{I}_n$ aus?

Anwendung: Blockdiagonale Matrizen

Kroneckerprodukt um Blockdiagonalmatrizen zu erzeugen:

$$\boldsymbol{I}_n \otimes_{\mathrm{K}} \underbrace{\boldsymbol{S}}_{p \times p} = \underbrace{\begin{pmatrix} \boldsymbol{S} & & & \\ & \boldsymbol{S} & & \\ & & \ddots & \\ & & & \boldsymbol{S} \end{pmatrix}}_{np \times np}$$

- Nützlich z.B. in Statistik für Kovarianzmatrizen von Vektoren mit unabhängigen Blöcken mit identischer Kovarianz
- **Achtung:**
  $\boldsymbol{I}_n \otimes_{\mathrm{K}} \boldsymbol{S}$ ist blockdiagonal, $\boldsymbol{S} \otimes_{\mathrm{K}} \boldsymbol{I}_n$ ist es i.A. *nicht* — auf die **Reihenfolge der Faktoren** kommt es an!

Anwendung: Separierbare Kovarianz

- z.B. Daten an m verschiedenen *Orten* zu n verschiedenen *Zeitpunkten* (Annahme hier: Daten geordnet Ort 1 Zeit 1, Ort 2 Zeit 1, Ort 1 Zeit 2, Ort 2 Zeit 2)
- *Modellannahme:* **Separierbare** Kovarianz mit $\mathbf{\Sigma} = \mathbf{\Sigma}_T \otimes_{\mathrm{K}} \mathbf{\Sigma}_S \in \mathbb{R}^{mn \times mn}$, wobei
  - $\mathbf{\Sigma}_T \in \mathbb{R}^{n \times n}$: Kovarianzmatrix zwischen Zeitpunkten
  - $\mathbf{\Sigma}_S \in \mathbb{R}^{m \times m}$: Kovarianzmatrix zwischen Orten
  - **Vorteile:** Parametersparsam: $mn(mn+1)/2$ vs. $m(m+1)/2 + n(n+1)/2$; einfachere Interpretation

Beispiel $m = 2$ Orte, $n = 2$ Zeitpunkte, $\mathbf{\Sigma}_T = \begin{pmatrix} 1.0 & 0.8 \\ 0.8 & 1.0 \end{pmatrix}$, $\mathbf{\Sigma}_S = \begin{pmatrix} 2.0 & 0.5 \\ 0.5 & 2.0 \end{pmatrix}$:

$$\mathbf{\Sigma} = \mathbf{\Sigma}_T \otimes_{\mathrm{K}} \mathbf{\Sigma}_S = \begin{pmatrix} 1.0 \cdot \mathbf{\Sigma}_S & 0.8 \cdot \mathbf{\Sigma}_S \\ 0.8 \cdot \mathbf{\Sigma}_S & 1.0 \cdot \mathbf{\Sigma}_S \end{pmatrix} = \begin{pmatrix} 2.0 & 0.5 & 1.6 & 0.4 \\ 0.5 & 2.0 & 0.4 & 1.6 \\ 1.6 & 0.4 & 2.0 & 0.5 \\ 0.4 & 1.6 & 0.5 & 2.0 \end{pmatrix} \in \mathbb{R}^{4 \times 4}$$

Interpretation:
Eintrag $(1,1)$: Varianz in Ort 1 zu Zeit 1 = 2.0; Eintrag $(1,3)$: Kovarianz zwischen Ort 1, Zeit 1 und Ort 1, Zeit 2 = $1.6 = 0.8 \cdot 2.0$

Anwendung: vec-Trick — Matrixgleichungen als LGS

Für $\boldsymbol{X} \in \mathbb{R}^{m \times n}$ stapelt $\operatorname{vec}(\boldsymbol{X}) = (x_{11}, \ldots, x_{m1}, x_{12}, \ldots, x_{mn})^\top \in \mathbb{R}^{mn}$ die Spalten von $\boldsymbol{X}$ zu einem Vektor.

vec-Trick

Für $\boldsymbol{A} \in \mathbb{R}^{k \times m}$, $\boldsymbol{X} \in \mathbb{R}^{m \times n}$, $\boldsymbol{B} \in \mathbb{R}^{n \times l}$ gilt

$$\operatorname{vec}(\boldsymbol{AXB}) = \left(\boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{A}\right) \operatorname{vec}(\boldsymbol{X}).$$

- Macht lineare **Matrix**gleichungen zu gewöhnlichen **LGS** $\boldsymbol{Mz} = \boldsymbol{c} \to$ lösbar mit den Methoden aus Kapitel *Gleichungssysteme* (LU, Cholesky, …)!
- Wichtige Spezialfälle ($\boldsymbol{I}_n$: Einheitsmatrix):

$$\operatorname{vec}(\boldsymbol{AX}) = (\boldsymbol{I}_n \otimes_{\mathrm{K}} \boldsymbol{A}) \operatorname{vec}(\boldsymbol{X}), \qquad \operatorname{vec}(\boldsymbol{XB}) = \left(\boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{I}_m\right) \operatorname{vec}(\boldsymbol{X})$$

- **Reihenfolge merken:**
  $\boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{A}$ und $\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B}^\top$ haben *immer* dieselbe Größe — ein Vertauschen fällt nicht durch unpassende Formate auf!

Bsp: vec-Trick löst die Sylvester-Gleichung

Gesucht:
$\boldsymbol{X} \in \mathbb{R}^{2\times 2}$ mit $\boldsymbol{AX} + \boldsymbol{XB} = \boldsymbol{C}$ *(z.B. stationäre Kovarianzen von Zeitreihenmodellen: Lyapunov-Gleichung)*

vec-Trick auf beide Summanden ($m = n = 2$):

$$\operatorname{vec}(\boldsymbol{AX} + \boldsymbol{XB}) = \left(\boldsymbol{I}_2 \otimes_{\mathrm{K}} \boldsymbol{A} + \boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{I}_2\right)\operatorname{vec}(\boldsymbol{X}) = \operatorname{vec}(\boldsymbol{C})$$

$\Longrightarrow$ ein ganz normales 4×4-LGS für $\operatorname{vec}(\boldsymbol{X})$:

```r
A <- matrix(c(3, 1, 0, 2), 2, 2)
B <- matrix(c(0, 1, 1, 1), 2, 2)
C <- matrix(c(1, 2, 3, 4), 2, 2)
# Systemmatrix des LGS via Kroneckerprodukte:
M <- kronecker(diag(2), A) + kronecker(t(B), diag(2))
X <- matrix(solve(M, as.vector(C)), 2, 2)
A %*% X + X %*% B                  # = C ?
all.equal(A %*% X + X %*% B, C)
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
[1] TRUE
```

- Ohne vec-Trick keine LGS-Form für unsere Löser aus Kap. 5 — mit: `solve()` genügt. *(Für große Matrizen gibt es effizientere spezialisierte Sylvester-Löser.)*

Tensorprodukt von Vektorräumen

Tensorprodukt von Vektorräumen

Def.: Tensorprodukt von Vektorräumen

Seien V und W zwei Vektorräume. Dann ist das **Tensorprodukt** $V \otimes W$ definiert als der Vektorraum

$$V \otimes W = \operatorname{span}\{\boldsymbol{v} \otimes \boldsymbol{w} \colon \boldsymbol{v} \in V, \boldsymbol{w} \in W\},$$

wobei $(\boldsymbol{v}, \boldsymbol{w}) \mapsto \boldsymbol{v} \otimes \boldsymbol{w}$ eine bilineare Abbildung ist.

- *Bilinearität* bedeutet: $(c\boldsymbol{v}) \otimes \boldsymbol{w} = c(\boldsymbol{v} \otimes \boldsymbol{w}) = \boldsymbol{v} \otimes (c\boldsymbol{w})$, $(\boldsymbol{v}_1 + \boldsymbol{v}_2) \otimes \boldsymbol{w} = \boldsymbol{v}_1 \otimes \boldsymbol{w} + \boldsymbol{v}_2 \otimes \boldsymbol{w}$ und $\boldsymbol{v} \otimes (\boldsymbol{w}_1 + \boldsymbol{w}_2) = \boldsymbol{v} \otimes \boldsymbol{w}_1 + \boldsymbol{v} \otimes \boldsymbol{w}_2$

Beispiel:

- Sei $V = \mathbb{R}^m$, $W = \mathbb{R}^n$ und $\otimes$ das äußere Produkt. Dann ist $V \otimes W = \mathbb{R}^m \otimes \mathbb{R}^n = \mathbb{R}^{m \times n}$.

Beispiel:

- Seien $V_1 = \mathbb{R}^m$, $V_2 = \mathbb{R}^n$, $V_3 = \mathbb{R}^q$ und $\otimes$ das Tensorprodukt. Dann ist

  $V_1 \otimes V_2 \otimes V_3 = \mathbb{R}^m \otimes \mathbb{R}^n \otimes \mathbb{R}^q = (\mathbb{R}^m \otimes \mathbb{R}^n) \otimes \mathbb{R}^q = \mathbb{R}^{m \times n} \otimes \mathbb{R}^q = \mathbb{R}^{m \times n \times q}$.

Bsp: $V \otimes W$ vs. $\boldsymbol{v} \otimes \boldsymbol{w}$

Wichtige Unterscheidung:

- $\boldsymbol{v} \otimes \boldsymbol{w}$: Ein einzelnes Tensorprodukt (Rang-1-Matrix für $\boldsymbol{v}, \boldsymbol{w} \neq \mathbf{0}$)
- $V \otimes W$: Der **gesamte Vektorraum**, der von allen solchen Produkten erzeugt wird

Beispiel: Sei $V = W = \mathbb{R}^2$

- $\boldsymbol{A} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ **nicht** als einzelnes Tensorprodukt $\boldsymbol{v} \otimes \boldsymbol{w}$ von $\boldsymbol{v}, \boldsymbol{w} \in \mathbb{R}^2$ darstellbar (Rang 2 $\neq$ 1).
- **Aber:** $\boldsymbol{A} \in V \otimes W = \mathbb{R}^{2\times 2}$:

$$\boldsymbol{A} = \boldsymbol{e}_1 \otimes \boldsymbol{e}_1 + \boldsymbol{e}_2 \otimes \boldsymbol{e}_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & 1 \end{pmatrix}$$
$$= \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \checkmark$$

Allgemein:

Jede Matrix $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ kann als Summe von höchstens $\min(m, n)$ Tensorprodukten geschrieben werden: $\boldsymbol{A} = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \otimes \boldsymbol{v}_i$ mit $r = \operatorname{rank}(\boldsymbol{A}) \rightarrow$ *Singulärwertzerlegung*!

Tensorproduktbasis für Vektorräume

Theorem

Sind $\mathcal{B}_V = \{\boldsymbol{v}_1, \dots, \boldsymbol{v}_m\}$ und $\mathcal{B}_W = \{\boldsymbol{w}_1, \dots, \boldsymbol{w}_n\}$ Basen von V bzw. W, dann ist

$$\mathcal{B}_{V\otimes W} = \{\boldsymbol{v} \otimes \boldsymbol{w} \colon \boldsymbol{v} \in \mathcal{B}_V, \boldsymbol{w} \in \mathcal{B}_W\}$$

eine Basis von $V \otimes W$ und wird **Tensorproduktbasis** genannt. Insbesondere gilt

$$V \otimes W = \operatorname{span}\left(\mathcal{B}_{V\otimes W}\right) = \left\{ \sum_{i=1}^{m} \sum_{j=1}^{n} c_{i,j} \boldsymbol{v}_i \otimes \boldsymbol{w}_j \colon c_{1,1}, \dots, c_{m,n} \in \mathbb{R} \right\}$$

und $\dim(V \otimes W) = \dim(V) \dim(W)$.

Bsp: Tensorproduktbasis für bivariate Polynome

Univariate Basis:
Raum der Polynome vom Grad ≤ 1 auf $[0,1]$: $\mathcal{P}_1 = \{p(t) = a_0 + a_1 t \colon a_0, a_1 \in \mathbb{R}\}$ mit Basis $\mathcal{B} = \{b_1, b_2\}$, $b_1(t) = 1, b_2(t) = t$.

Tensorproduktbasis für bivariate Funktionen $f \colon [0,1] \times [0,1] \to \mathbb{R}$:
Basisfunktionen $\phi_{jk} = b_j \otimes b_k$ mit $(b_j \otimes b_k)(x,y) = b_j(x) \cdot b_k(y)$, also

$$\begin{aligned} \phi_{11}(x,y) &= b_1(x)b_1(y) = 1 \quad ; \quad \phi_{12}(x,y) = b_1(x)b_2(y) = y \\ \phi_{21}(x,y) &= b_2(x)b_1(y) = x \quad ; \quad \phi_{22}(x,y) = b_2(x)b_2(y) = xy \end{aligned}$$

Approximation einer bivariaten Funktion:
Z.B. $f(x,y) = 2 - x + 3y + 5xy = 2\phi_{11}(x,y) + 3\phi_{12}(x,y) - 1\phi_{21}(x,y) + 5\phi_{22}(x,y)$

Dimension: $\dim(\mathcal{P}_1 \otimes \mathcal{P}_1) = 2 \cdot 2 = 4$ ✓

Allgemein: Für k-variate Tensorprodukt-Polynome vom Grad $\leq d$ *in jeder Variablen* (univariat: $\mathcal{P}_d = \left\{p(t) = \sum_{r=0}^{d} a_r t^r \colon a_r \in \mathbb{R}\right\}$): $\dim(\mathcal{P}_d^{\otimes k}) = (d+1)^k$

Anwendung in der Praxis:

- **Statistik:** Tensorprodukt-Splines für multivariate Regression (GAMs) — ausführlich im Kapitel *Funktionsapproximation*
- **Finite-Elemente-Methoden (FEM):** Approximation von PDEs mit tensorbasierten Polynomen
- **Numerik:** Sparse Grids vermeiden die exponentielle Anzahl $(d+1)^k$ durch geschickte Auswahl

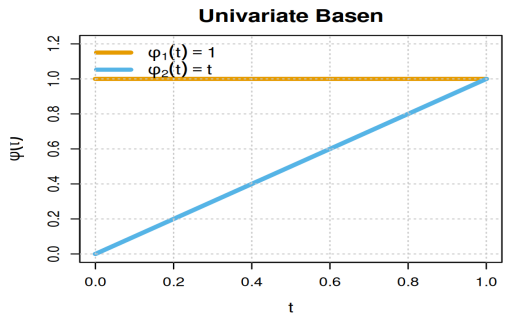
Bsp: Tensorproduktbasis für bivariate Polynome

Tensorproduktbasis

$P_1 \times P_1$

Univariate Basen:

$B_x = \{1, x\}$



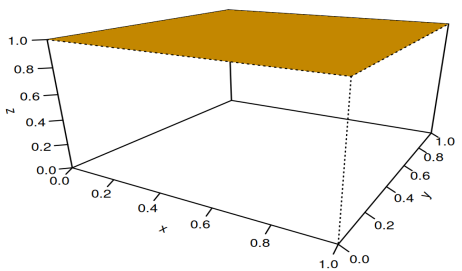
$B_y = \{1, y\}$

Tensorprodukt:

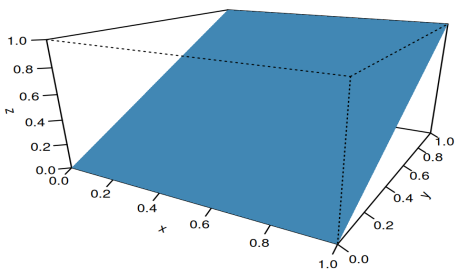
$B_x \times B_y$

erzeugt 4 bivariate

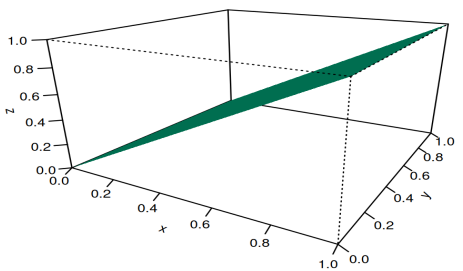
$\varphi_1 = 1 \times 1$



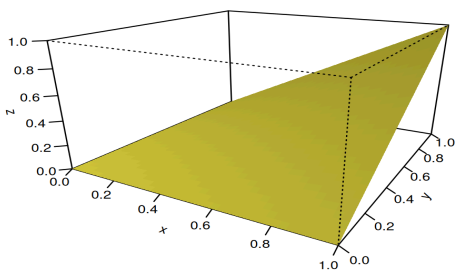
$\varphi_2 = 1 \times y$



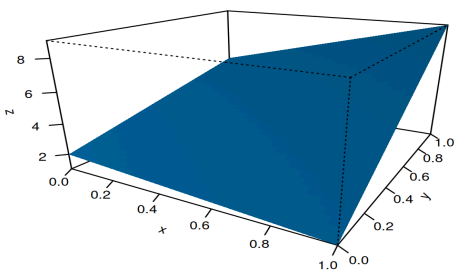
$\varphi_3 = x \times 1$



$\varphi_4 = x \times y$



Beispiel: $f(x, y) = 2 - x + 3y + 5xy$



Anwendung: Tensorprodukt-Splines — Designmatrix

Vorgriff aufs Kapitel *Funktionsapproximation*: flexible univariate Funktionen via *B-Spline*-Basis $B_1, \ldots, B_K$; bivariat via *Tensorproduktbasis* $\phi_{jk}(x, y) = B_j(x) B_k(y)$.

- Auswertung von $f(x, y) = \sum_{j,k} c_{jk} \phi_{jk}(x, y)$ auf einem **Gitter** aus x- und y-Werten:

$$\underbrace{\boldsymbol{F}}_{f(x_i, y_l)} = \boldsymbol{B}_x \boldsymbol{C} \boldsymbol{B}_y^\top$$

  mit univariaten Basismatrizen $\boldsymbol{B}_x \in \mathbb{R}^{n_x \times K}$, $(\boldsymbol{B}_x)_{ij} = B_j(x_i)$ (analog $\boldsymbol{B}_y$)

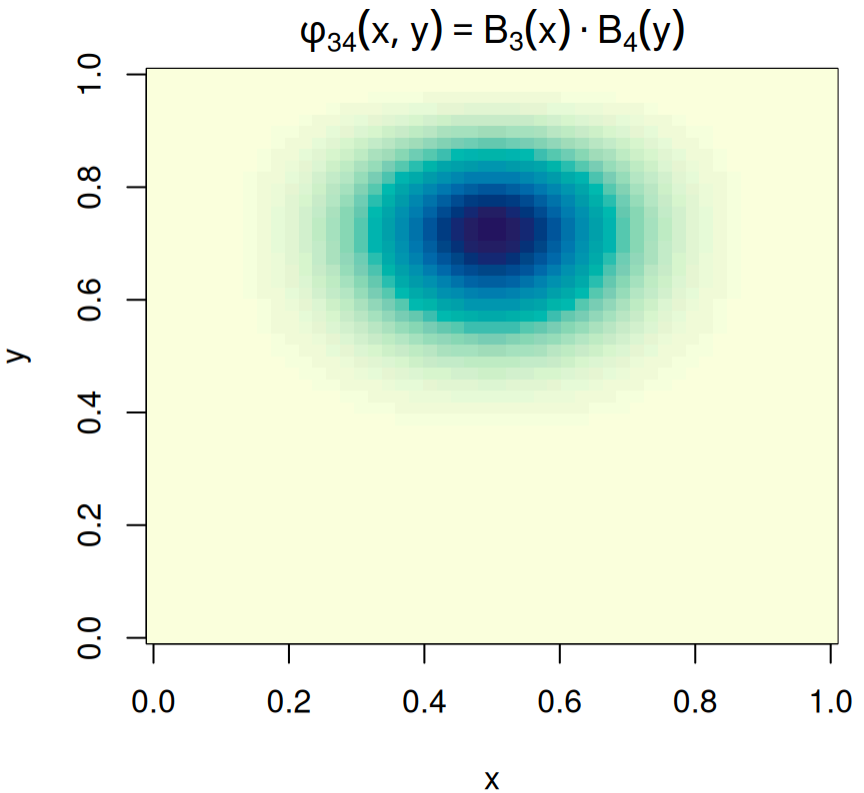
- **vec-Trick!**

$$\operatorname{vec}(\boldsymbol{F}) = (\boldsymbol{B}_y \otimes_{\mathrm{K}} \boldsymbol{B}_x) \operatorname{vec}(\boldsymbol{C})$$

  $\Longrightarrow$ Designmatrix des Gitters ist das **Kroneckerprodukt** $\boldsymbol{B} = \boldsymbol{B}_y \otimes_{\mathrm{K}} \boldsymbol{B}_x$

```r
library(splines)
x <- seq(0, 1, length.out = 50)
Bx <- bs(x, df = 6)  # B-Spline-Basis
By <- bs(x, df = 6)
B <- kronecker(By, Bx)
dim(B)  # 2500 Gitterpunkte, 6*6 Koeffizienten
```

```
[1] 2500   36
```



Eine bivariate Basisfunktion = äußeres Produkt zweier univariater B-Splines. → Kap. *Funktionsapproximation I/II*: gute univariate Basen $\otimes$-kombinieren!

Self-Check & Zusammenfassung

Kernkonzepte & Anwendungen

- *Tensoren*: höher-dimensionale Verallgemeinerung von Vektoren und Matrizen
  - Extrem wichtig für Deep Learning, Bild-/Videobearbeitung, moderne Statistik
- *Multilinearität* vs Linearität:
  - Linear: $f(a\boldsymbol{v} + b\boldsymbol{w}) = af(\boldsymbol{v}) + bf(\boldsymbol{w})$ vs. Multilinear:
    $f(\boldsymbol{v}_1, \ldots, a\boldsymbol{v}_i + b\boldsymbol{w}_i, \ldots, \boldsymbol{v}_n) = af(\boldsymbol{v}_1, \ldots, \boldsymbol{v}_i, \ldots, \boldsymbol{v}_n) + bf(\boldsymbol{v}_1, \ldots, \boldsymbol{w}_i, \ldots, \boldsymbol{v}_n)$
  - Bsp: Matrixmultiplikation bilinear aber nicht linear.
- *Äußeres Produkt*: Erzeugt (Rang-1-)Matrizen aus Vektoren ($\rightarrow$ *SVD*)
- *Tensorprodukt*: Verallgemeinerung des äußeren Produkts auf Tensoren höherer Stufe
- *Kroneckerprodukt* $\otimes_{\mathrm{K}}$: Spezielle Variante des Tensorprodukts von Matrizen ($\otimes$)
  - **vec-Trick** $\mathrm{vec}(\boldsymbol{AXB}) = (\boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{A})\mathrm{vec}(\boldsymbol{X})$: Matrixgleichungen (Sylvester/Lyapunov) $\rightarrow$ gewöhnliche LGS
  - Statistik: separierbare/Kronecker-strukturierte Kovarianzen, blockdiagonale Matrizen
- *Tensorprodukte bauen komplexe Räume auf*:
  - Konstruiere multivariate (Funktions-)Räume aus univariaten — Designmatrix auf Gittern = Kroneckerprodukt $\rightarrow$ **Funktionsapproximation**, (Höherdimensionale) *Splines*

Nächste Vorlesungen:
Fortgeschrittene Differentialrechnung

Self-Check

Frage 1: Daten an $m = 10$ Orten zu $n = 20$ Zeitpunkten. Eine unstrukturierte Kovarianzmatrix des 200-dimensionalen Datenvektors hat $\frac{200 \cdot 201}{2} = 20100$ freie Parameter. Wie viele Parameter haben die beiden Faktoren $\boldsymbol{\Sigma}_T, \boldsymbol{\Sigma}_S$ des separierbaren Modells $\boldsymbol{\Sigma} = \boldsymbol{\Sigma}_T \otimes_{\mathrm{K}} \boldsymbol{\Sigma}_S$ zusammen?

a) 265 **b)** 11550 **c)** 20100 **d)** 200

Frage 2: Sei $f\colon \mathbb{R}^2 \times \mathbb{R}^2 \to \mathbb{R}$ bilinear. Was ist $f(2\boldsymbol{v}, 3\boldsymbol{w})$?

a) $2f(\boldsymbol{v}, \boldsymbol{w})$ **b)** $5f(\boldsymbol{v}, \boldsymbol{w})$ **c)** $6f(\boldsymbol{v}, \boldsymbol{w})$ **d)** $f(\boldsymbol{v}, \boldsymbol{w})$

Frage 3: Der vec-Trick besagt $\mathrm{vec}(\boldsymbol{AXB}) = \,?$

a) $(\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B})\,\mathrm{vec}(\boldsymbol{X})$ **b)** $(\boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{A})\,\mathrm{vec}(\boldsymbol{X})$ **c)** $(\boldsymbol{B} \otimes_{\mathrm{K}} \boldsymbol{A}^\top)\,\mathrm{vec}(\boldsymbol{X})$ **d)** $(\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B}^\top)\,\mathrm{vec}(\boldsymbol{X})$

Anhang

Anwendung: RGB-Bilder als Tensoren

Beispiel 1: RGB-Bild

- Farbbild der Größe 224×224 Pixel wird als Tensor $\boldsymbol{I} \in \mathbb{R}^{224 \times 224 \times 3}$ dargestellt:
  - $\boldsymbol{I}_{i,j,1}$: Rot-Intensität an Position (i, j) (Wert zwischen 0 und 255)
  - $\boldsymbol{I}_{i,j,2}$: Grün-Intensität an Position (i, j)
  - $\boldsymbol{I}_{i,j,3}$: Blau-Intensität an Position (i, j)
- Einträge: $224 \times 224 \times 3 \approx 150 \cdot 10^3$ Werte pro Bild



$\simeq$



↩ zurück: „Tensoren in Deep Learning"

Anwendung: Batch-Tensoren in CNNs

Beispiel 2: Batch von Bildern (Convolutional Neural Networks) (CNNs)

- CNN verarbeitet typischerweise mehrere Bilder gleichzeitig: $\boldsymbol{B} \in \mathbb{R}^{32\times224\times224\times3}$
  1. Dimension (Batch-Size): 32 Bilder gleichzeitig
  2. Dimension (Höhe): 224 Pixel
  3. Dimension (Breite): 224 Pixel
  4. Dimension (Kanäle): 3 Farbkanäle (RGB)
- Einträge: $32 \times 224 \times 224 \times 3 \approx 4.8 \cdot 10^6$ Werte

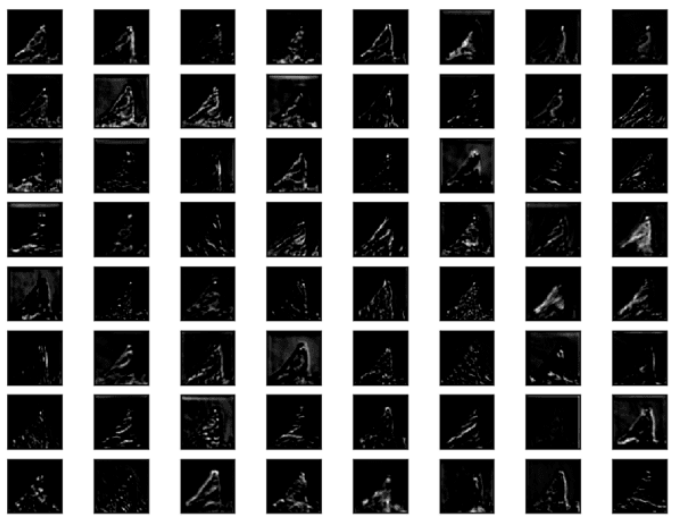
Anwendung: Feature Maps in CNNs

Beispiel 3: Feature Maps in CNNs

- Convolutional Layer c: $\mathbb{R}^{32\times224\times224\times3} \to \mathbb{R}^{32\times56\times56\times64}$
- Reduziert (üblicherweise) räumliche Auflösung $(224 \times 224 \to 56 \times 56)$
- Erhöht Anzahl der “Kanäle” (*features*) $(3 \to 64)$



$\to$



Feature Map: Block 3 des VGG16-Modells; Quelle: machinelearningmastery.com

↩ zurück: „Tensoren in Deep Learning”

Universelle Eigenschaft des Tensorprodukts

- Das *Tensorprodukt* $V \otimes W$ hat eine *universelle Eigenschaft*: **Jede** bilineare Abbildung $f\colon V \times W \to Z$ kann eindeutig als lineare Abbildung $\tilde{f}\colon V \otimes W \to Z$ faktorisiert werden:

$$f(\boldsymbol{v}, \boldsymbol{w}) = \tilde{f}(\boldsymbol{v} \otimes \boldsymbol{w}) \quad \text{für alle } \boldsymbol{v} \in V, \boldsymbol{w} \in W$$

- Diese Eigenschaft macht das Tensorprodukt zur “natürlichen” Konstruktion für die Linearisierung bilinearer Abbildungen.